Enfoque Modular y Diseño Top-Down

Calculadora de área de un círculo con funciones modulares

Alexis Maier

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B

Profesor: Prof. Mansilla Muñoz York Elías

Fecha: 01 de octubre de 2026

Introducción

La actividad 3 abordó el diseño top-down aplicado a un problema matemático clásico: el cálculo del área de un círculo. La consigna no se limitaba a escribir la fórmula dentro del programa, sino que exigía que el cálculo residiera en un módulo independiente, declarado mediante prototipo antes de *main()* y definido al final del archivo. Se agregó además un mecanismo de validación del dato geométrico y un bloque de gamificación que dibuja en la consola una diana de tiro en arte ASCII cuyo tamaño depende del radio ingresado.

Desarrollo

Diseño top-down del programa

El algoritmo se resolvió partiendo del resultado que se quería obtener y degradando la problemática hasta llegar a operaciones elementales. El nivel superior corresponde a la función *main()*: presentar el programa, solicitar el radio, validar la entrada, invocar el módulo de cálculo y mostrar el resultado. El segundo nivel es el módulo *calcularAreaCirculo(double)*, que encapsula la única operación matemática necesaria. Cada nivel invoca al inferior sin conocer su implementación interna, lo que permite modificar la fórmula sin tocar la interfaz.

Prototipo y definición

Antes de *main()* se declara el prototipo *double calcularAreaCirculo(double radio);*, que informa al compilador de la firma de la función y de su tipo de retorno. Esta declaración permite invocar el módulo sin desplazar la implementación, ya que el enlace se resuelve en tiempo de compilación. La definición completa, con el cuerpo delimitado por llaves, aparece al final del archivo.

Implementación del cálculo

El módulo declara la constante *const double PI = 3.1415926535;* dentro de su propio ámbito local y aplica la fórmula geométrica *area = PI * (r * r)*. Al ser *const*, el compilador garantiza que el valor no sea alterado y puede optimizar el cálculo. El resultado se devuelve con *return* y se recibe en *main()* dentro de la variable *areaFinal*.

Validación del dato

Un radio igual o menor a cero no describe ningún círculo y produce un área sin sentido físico. El programa detecta esa condición antes de invocar al módulo, imprime un mensaje de error claro y termina con *return 1;*, de modo que nunca se ejecuta un cálculo sobre un dato inválido.

Bloque de gamificación

Como ampliación del enunciado, la consola renderiza una diana de tiro en arte ASCII cuyo tamaño se selecciona según tres rangos de radio: compacto para radios de hasta 5 cm, estándar hasta 12 cm y gigante por encima de ese valor. La estructura *if* / *else if* / *else* selecciona el dibujo correspondiente, divulgando el resultado sin alterar la lógica principal del cálculo.

Compilación y evidencia

El módulo se compiló con *g++ "src/Act 3 - calculo_area.cpp" -o src/calculadoraarea.exe* y se almacenaron las capturas de compilación y del diagrama de flujo en *capturas/*.

Cuestionario Técnico de Control

1. Utilidad del prototipo

El prototipo es una declaración que anticipa la firma de una función. Su principal beneficio es la legibilidad: permite agrupar las declaraciones al inicio del archivo y ubicar la implementación al final, que es el estilo exigido por la consigna. Además, los eventuales errores de firma se detectan en la etapa de compilación y no durante la ejecución.

2. Parámetros y valor de retorno

El parámetro *radio* es la información que el módulo necesita para operar y se recibe por valor: la función trabaja sobre una copia y no altera el dato original del llamador. El valor de retorno, de tipo *double*, es la información que el módulo entrega; ese resultado se asigna a *areaFinal* para poder mostrarse por pantalla.

3. Significado de return 1

Finalizar el programa con un código distinto de cero señala al sistema operativo que la ejecución no fue satisfactoria. En este caso el dato geométrico era inválido, de manera que el corte temprano evita calcular con un radio negativo y comunica con claridad al usuario la causa del fallo.

4. Precisión del tipo numérico

La precisión depende del tipo numérico empleado. Un *float* almacena alrededor de 7 dígitos significativos y un *double* cerca de 15, por lo que para cálculos que involucran potencias de números decimales conviene utilizar *double*. Constantes como PI deben escribirse siempre con decimales, porque la división entre enteros truncaría el resultado hacia cero.

5. Autoevaluación

La mayor dificultad fue recordar la declaración del prototipo antes de la definición, pues es una costumbre propia de C++ que no aparece en otros lenguajes del curso. También resultó relevante la validación previa, porque obliga a decidir qué ocurre con las entradas que rompen la hipótesis del problema.

Conclusión

La actividad mostró cómo el diseño top-down convierte un problema numérico en un programa mantenible. Separar el cálculo en un módulo con nombre propio permite testear la fórmula de forma aislada, reutilizarla en otros programas y entender que la complejidad puede distribuirse entre varias funciones bien delimitadas. La validación del radio enseñó que un programa no termina cuando calcula un resultado, sino cuando también sabe comunicar qué debe hacer cuando no puede calcularlo. En conjunto, la actividad sentó las bases de la modularidad que necesitarán las estructuras de datos y la programación orientada a objetos de los talleres siguientes.

Referencias

cppreference.com. (s. f.). *Functions*. Referencia del lenguaje C++. https://en.cppreference.com/w/cpp/language/function

cplusplus.com. (s. f.). *Functions*. Tutorial de C++. https://cplusplus.com/doc/tutorial/functions/

Microsoft Learn. (s. f.). *Introducción a los módulos de programación*. https://learn.microsoft.com/es-es/cpp/

GeoGebra. (s. f.). *Calculadora de áreas*. Material didáctico de geometría. https://www.geogebra.org/